

What is DOM?

- The DOM (Document Object Model) is like a family tree of your HTML page.
- represents structure of webpage as objects that JS can interact with.
- HTML is like house blueprint
- DOM is actual house, ~~is~~ that can be modified.

→ Javascript is like having tools to modify, add new element.

<html>

<body>

<h1> Website </h1>

<p> Hello World </p>

</body>

</html>

DOM Tree:

html

└ body

└ h1

└ p

Selecting Element

1. document.getElementById()

→ Gets one element by its unique ID

// HTML: <div id="myDiv">Hello</div>

```
let element = document.getElementById  
('myDiv');
```

```
console.log(element.textContent);
```

2. document.getElementsByClassName()

→ Gets multiple elements by class name

(return HTMLCollection)

```
let elements = document.getElementsByClassName  
('highlight');
```

```
console.log(elements.length);
```


3. getElementByTagName()

```
let paragraphs = document.getElementsByTagName("p");
```

```
for (let i = 0; i < paragraphs.length; i++)
```

```
{
```

```
    console.log(paragraphs[i].  
        textContent);
```

```
}
```

4. querySelector()

→ Gets the first element that matches CSS selector

```
let element = document.querySelector  
("#myDiv");
```


// By ID

```
let element = document.querySelector('highlight');
```

// By Tag

```
let element = document.querySelector('p');
```

// Complex selector

```
let element = document.querySelector('div > p.special');
```

5. querySelectorAll()

→ Gets all elements that match CSS selector (return NodeList)

```
let element = document.querySelectorAll  
( '.highlight');
```


Changing Content

```
let div = document.querySelector('#myDiv');
```

// innerHTML - Gets / sets HTML content

```
div.innerHTML = '<strong> Bold text </strong>';
```

```
console.log(div.innerHTML);
```

// textContent - Gets / sets text only

```
div.textContent = '<strong> Bold text </strong>';
```

```
console.log(div.textContent);
```

// innerText - Similar to textContent but respects styling (hidden element)

```
div.innerText = 'Visible text only';
```


innerHTML: When you need to add
HTML tags

textContent When you want plain
text (safer, prevent XSS attack)

innerText: When you need to
respect styling.

Changing Styles

Using .style property

```
let element = document.querySelector
```

```
(#myDiv);
```

```
element.style.color = 'red';
```

```
element.style.cssText = 'color: blue;
```

```
padding 10px';
```


Adding/Removing CSS classes

```
let element = document.querySelector('#myDiv');  
element.classList.add('highlight');
```

```
element.classList.remove('hidden');
```

Working with Attribute

```
let element = document.querySelector('#myInput');
```

// Get Attribute

```
let value = element.getAttribute('placeholder');  
element.setAttribute('placeholder', 'Enter.....');
```

```
element.removeAttribute('disable');
```


// Check if attribute exists

```
if (element.hasAttribute('require'))  
div); { console.log('Input is required'); }
```

// For common attribute use direct properties

```
element.id = 'newId';
```

```
element.className = 'newClass';
```

```
put); element.value = 'new value';
```

Creating and Adding Elements

```
le'); // Create new Element
```

```
}; let newDiv = document.createElement  
('div');
```

```
newDiv.textContent = 'I am new';
```

```
newDiv.className = 'new-element';
```


// Add to page

```
let container = document.querySelector.  
    ('#container');
```

```
container.appendChild(newDiv);
```

// Insert before existing element

```
let newElement = document.createElement('p');
```

```
newElement.textContent = 'I come first';
```

```
container.insertBefore(newElement, existingElement);
```

// Remove element

```
let elementToRemove = -----
```

```
elementToRemove.parentNode.removeChild;
```

// Modern way

```
elementToRemove.remove();
```


DOM Tree Traversal

```
let element = document.querySelector(the element);
```

// Going up the tree

```
let parent = element.parentNode;
```

```
let parentElement = element.parentElement;
```

// Going Down the tree

```
let children = element.childNodes;
```

```
let childElements = element.children;
```

```
let firstChild = element.firstChild;
```

```
let lastChild = element.lastElementChild;
```

// Going sideways

```
let nextSibling = element.nextSibling;
```

```
let prevSibling = element.previousElement  
sibling;
```


// Example of Navigation through list

```
let listItem = document.querySelector('li');  
let nextItem = listItem.nextElementSibling;  
let list = listItem.parentElement;
```

What are Events?

Events are the things that happen in HTML when users interact with webpages

- User clicks a button → `click` event
- User types in input → `keydown` event
- Page finishes loading → `load` event

// Click event

```
button.addEventListener('click', function(){  
    console.log('Button clicked');  
});
```


// Mouse Event

element.addEventListener

```
{  
  ('mouseover', function(){  
    console.log('Mouse over');  
  });  
}
```

Inline vs addEventListener()

egs. Inline: <button onclick = "alert()">

Limited to one handlers

addEventListener()

handle multiple time

Flow of event propagation

Event Propagation has three phases:

Capture Phase: Event goes down from document to target

Target phase: Event reaches the target element

Bubble phase: Event bubbles up from target to document

preventDefault()

Stops the default browser behavior of an event,

Example: Preventing from submission to handle it

event Delegation

Event delegation is attaching a single event listener to a parent element to handle events from child

elements. Useful for

→ Better Performance with many element

→ Handling dynamically created elements

→ less memory usage.

Form Submission handled

Use the `submit` event with

`preventDefault()` to stop page

reload, then access form

data using `FormData` or

element values to process with

JS.

Best Practices for DOM manipulation

- Minimize DOM queries
- use eventContent for plain text
- Batch DOM changes to avoid reflows
- Use CSS classes instead of inline styles